Otimização de Acesso

Tuesday, April 21, 2026 3:06 PM

Requisitos ideias de memória (principal)

- Tamanho ilimitado
- Acesso instantâneo para leitura e escrita

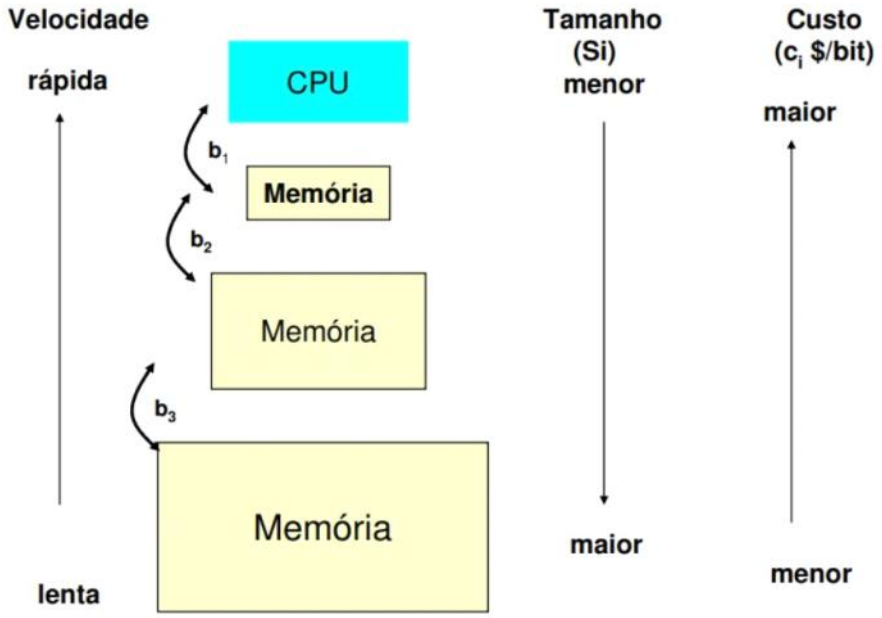
Na prática, esses requisitos são contraditórios

- **Quanto maior a memória, maior será o tempo de acesso**

Solução para a contradição: Criar uma **"ilusão"** para o processador de forma que a memória pareça ser grande e rápida o suficiente para não perceber, ou perceber minimamente, a existência de um gargalo no sistema.

1. A Pirâmide da Hierarquia

O sistema organiza as memórias em níveis. No topo, temos pouquíssima memória, mas na velocidade da CPU. Na base, temos memória massiva, mas a CPU levaria "eras" para acessar se dependesse apenas dela.



2. O Princípio da Localidade

A ilusão só funciona porque os programas não acessam a memória de forma aleatória. Eles seguem padrões:

- **Localidade Temporal:** Se a CPU acessou um dado agora, há uma chance enorme de ela precisar dele de novo em breve (ex: uma variável dentro de um loop).
- **Localidade Espacial:** Se a CPU acessou o endereço `1000`, é muito provável que o próximo dado que ela queira esteja no `1001` (ex: percorrendo um array ou as próximas linhas de

código).

3. Acesso Transparente e Cache

Quando a CPU pede um dado, ela sempre olha primeiro no nível mais alto (Cache L1). Se não estiver lá (**Miss**), o hardware busca no nível abaixo (L2, L3, RAM) e traz não apenas o dado pedido, mas um **bloco inteiro** de dados vizinhos para a Cache.

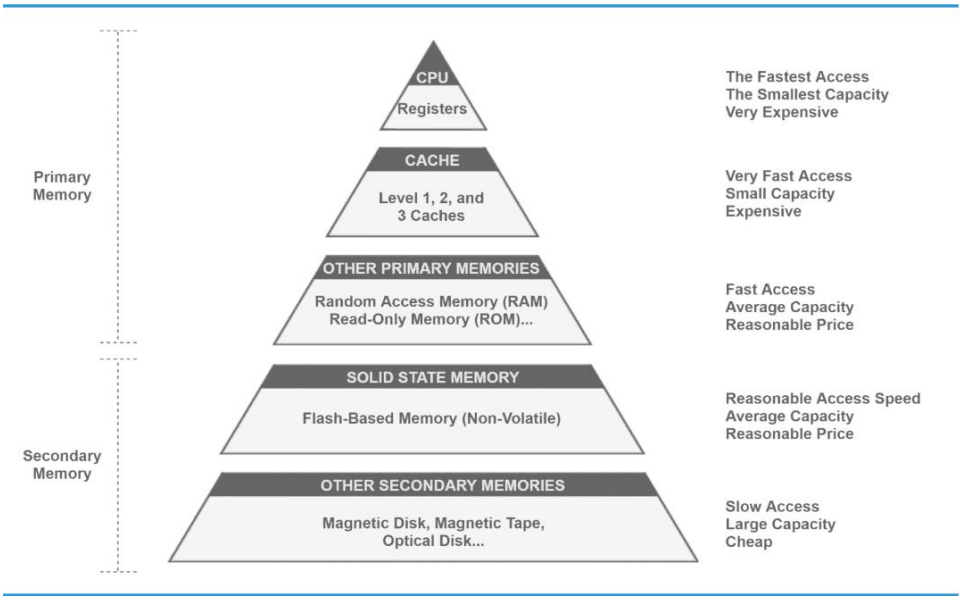
Isso é "transparente" porque o programador não precisa dizer "mova o dado da RAM para a Cache". O hardware faz isso sozinho. A CPU quase sempre encontra o que precisa nos níveis rápidos, criando a percepção de que todo o sistema opera naquela velocidade.

4. O Custo do "Miss" (A Falha da Ilusão)

A ilusão "quebra" quando ocorre um **Cache Miss** ou, pior, um **Page Fault** (quando o dado não está nem na RAM e precisa vir do disco).

- Acessar um registrador leva **<1ns**.
- Acessar a RAM leva cerca de **100ns**.
- Acessar um disco (SSD/HD) pode levar de **10.000ns a milissegundos**.

Sem a hierarquia e a cache, o processador passaria 99% do seu tempo parado, esperando os dados chegarem dos componentes mais lentos.

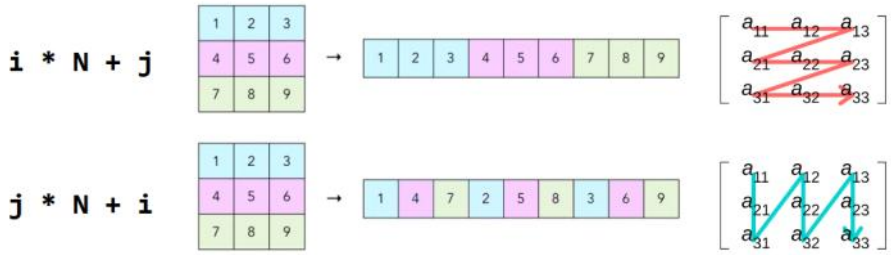


Princípio Localidade

Tuesday, April 21, 2026 3:18 PM

```c
int main()
{
    double A[N][N];
    int i, j;

    for (__ = 0; __ < N; __++)
        for (_ = 0; _ < N; _++)
            A[i][j] = 0.01;
    return 0;
}
```



No código apresentado, qual a diferença de escolha das variáveis de controle como sendo interna ou externa?

- Diz respeito a "contiguidade" (estado do que é adjacente, próximo ou vizinho, indicando contato físico imediato ou proximidade no tempo) do acesso à memória
- Com a escolha correta, as posições de memória são acessadas de maneira subsequente (algo que ocorre logo após, posteriormente, ou na sequência de outra coisa no tempo ou espaço)
- E por consequência, cache pode ser aproveitada
- Ou seja: A escolha correta é a que ocorre na sequência por causa da sua proximidade na memória

Memória Cache

Tuesday, April 21, 2026 3:28 PM

Memória cache é a solução técnica para o "gargalo" (aquilo que limita) de desempenho entre a CPU e a memória principal (RAM).

1. Definições Técnicas Essenciais

- **Bloco:** A unidade mínima de informação que é movida entre os níveis de memória. A cache não traz apenas um dado, mas um bloco de vizinhos.
- **Hit (Acerto):** Quando a CPU procura um dado e ele **está** na cache. É o cenário ideal.
- **Miss (Falha):** Quando o dado **não está** na cache e precisa ser buscado na RAM, gerando uma perda de tempo (penalidade).
- **Hit-Rate / Miss-Rate:** A porcentagem de vezes que a CPU encontra (ou não) o dado na cache.

2. O Funcionamento da "Ilusão"

A cache funciona criando uma ilusão de que a memória é **gigante** (como o HD) e **instantânea** (como os registradores).

- **Velocidade:** Enquanto a RAM leva cerca de **100 ns** para responder, a cache L1 leva apenas **10 ns** (e os registradores apenas **1 ns**).
- **Transparência:** O hardware gerencia a cache sozinho. O programa solicita um endereço de memória e o sistema de cache intercepta o pedido para ver se o dado já está lá.

3. A Lógica por trás: Princípio da Localidade

A cache só é eficiente porque os programas possuem comportamentos previsíveis:

- **Localidade Temporal:** Se um dado foi usado agora, ele provavelmente será usado de novo em breve (ex: variáveis de um loop ou contadores).
- **Localidade Espacial:** Se um dado foi usado, os dados que moram "vizinhos" a ele na memória provavelmente serão usados em seguida (ex: elementos de um array ou as próximas instruções do código).

4. Desempenho e o Cálculo de AMAT

O desempenho real da memória é medido pelo **AMAT (Average Memory Access Time)**. A fórmula apresentada nos slides é:

$$AMAT = HitTime + MissRate * MissPenalty$$

- **HitTime:** Tempo para acessar a cache.
- **MissRate:** Frequência de falhas (1 - HitRate).
- **MissPenalty:** O tempo extra gasto para buscar o dado na RAM quando ele não está na cache

Desempenho de Memória

Tuesday, April 21, 2026 3:49 PM

1. Taxa de Acerto (Hit-Rate)

Mede a eficácia da cache em encontrar o dado solicitado.

$$HitRate = \frac{\text{Número de Hits}}{\text{Número Total de Acessos}}$$

2. Taxa de Falha (Miss-Rate)

Mede a frequência com que o dado não é encontrado na cache. É o inverso da taxa de acerto.

$$MissRate = 1 - HitRate$$

ou

$$MissRate = \frac{\text{Número de Misses}}{\text{Número Total de Acessos}}$$

3. Tempo Médio de Acesso à Memória (AMAT)

Esta é a fórmula principal para calcular o desempenho real do sistema. Ela combina a velocidade da cache com a penalidade da lentidão da RAM.

$$AMAT = \text{HitTime} + (\text{MissRate} \times \text{MissPenalty})$$